

Nome: João Geiger Piza

Resumo explicando o que é Transfer Learning:

Transfer Learning é uma técnica em que um modelo aproveita conhecimentos já aprendidos em uma tarefa anterior para resolver uma nova tarefa relacionada, em vez de começar do zero. Na prática, isso é muito útil quando há poucos dados rotulados ou quando treinar um modelo grande seria caro em tempo e recursos.

Diferença entre extração de características e fine-tuning:

Extração de características e fine-tuning são duas formas de usar um modelo pré-treinado. Na extração, as camadas base ficam congeladas e só o novo classificador é treinado. Já no fine-tuning, algumas camadas do modelo base também são descongeladas e ajustadas para o novo problema. Em geral, a extração é mais simples e rápida, enquanto o fine-tuning adapta melhor o modelo quando o novo dataset é mais parecido com o original e há dados suficientes.

Importância do pré-treinamento em Deep Learning:

A importância do pré-treinamento em Deep Learning está em fornecer ao modelo representações úteis antes da tarefa final, reduzindo a necessidade de treinar tudo do zero. Isso costuma diminuir o custo computacional, acelerar o treinamento e melhorar a generalização, especialmente quando o conjunto de dados novo é pequeno.

Resumo das anotações do Capítulo 8:

O capítulo apresenta o fine-tuning como uma forma de transferência baseada em parâmetros, explica por que redes profundas podem ser transferidas, discute como fazer pré-treinamento e ajuste fino, a necessidade de regularização e lida com questões como “quantas camadas congelar ou ajustar”. Este também destaca que camadas mais baixas tendem a aprender características mais gerais, enquanto as mais altas capturam padrões mais específicos, e que a similaridade entre os domínios influencia fortemente o desempenho da transferência.

Descrição dos experimentos realizados com PyTorch:

Os experimentos realizados, de forma breve, envolvem a análise da arquitetura ResNet18 e a utilização de pesos pré-treinados no conjunto ImageNet para a tarefa de classificação do MNIST, observando acurácia, F1-score e as curvas de perda e acurácia.

Respostas e reflexões sobre os exercícios práticos:

1) Tanto o modelo original quanto o resnet18 apresentaram curvas de custo e acurácia no treinamento bem similares, com mudanças relativamente similares entre as épocas e velocidades de convergência quase idênticas. Porém, ao observar as curvas de custo e acurácia na validação, podemos observar que as do resnet18 apresentam comportamentos extremamente similares às de treinamento, indicando que o modelo realmente está aprendendo, enquanto as do modelo original, após a segunda época, começam a divergir da curva de treinamento, podendo indicar problemas como overfitting. Apesar disto, a acurácia final no conjunto de teste de ambos os modelos foi bem similar (original foi 98.92% e o resnet18 foi 98.66%).

2) Ao usar o modelo resnet18 com os pesos pré-treinados do ImageNet, sem nenhum treinamento, foram obtidos uma acurácia no conjunto de teste de 71.57%

no ImageNet e 16.45% no MNIST. Ou seja, ocorreu o esperado, que era que o modelo fosse muito mais preciso no ImageNet, pois os pesos já foram treinados baseados neste, do que no MNIST, que apresentou acurácia ligeiramente melhor do que simplesmente tentar adivinhar qual classe é. O resultado mostra que os padrões aprendidos no domínio de origem não se generalizam bem para um domínio muito diferente, já que o ImageNet contém imagens coloridas, complexas e variadas de objetos do mundo real, enquanto o MNIST possui imagens simples compostas apenas por dígitos manuscritos. Assim, as características visuais aprendidas pela resnet18 durante o treinamento no ImageNet não conseguem representar adequadamente os padrões do MNIST sem adaptação adicional.

3)

- 3.1) Os parâmetros foram congelados usando um for loop para iterar entre os parâmetros das camadas e definir `requires_grad` para `False`, desativando o cálculo dos gradientes destes, da seguinte forma:

```
for param in model.parameters():  
    param.requires_grad = False
```

Para descongelar, foi utilizado o mesmo for loop com 2 mudanças: especificação da camada a ser descongelada e a mudança do `False` por `True`. A camada final foi substituída para se adequar ao número de classes do MNIST da seguinte forma:

```
model.fc = nn.Linear(model.fc.in_features, 10)
```

Assim, a saída é mudada para 10, que é o número de classes do MNIST.

- 3.2) Faz sentido ajustar apenas as camadas finais da rede pois as camadas iniciais de uma rede neural costumam aprender padrões bem gerais, como bordas, texturas e formas simples, que servem para muitas tarefas diferentes. Já as camadas finais aprendem características mais específicas da tarefa original. Logo, ao ajustar apenas as camadas finais, podemos manter a identificação dos padrões gerais, mas podemos moldar o modelo para aprender as características da nova tarefa.

- 3.3) O treinamento, em geral, produziu resultados consideravelmente melhores que as outras estratégias. Isto era esperado, já que treinamos mais camadas em torno do dataset MNIST do que os outros.

- 5) A melhor estratégia foi o treinamento do zero, que obteve os maiores valores de acurácia e F1 no teste. Entre as estratégias com pesos pré-treinados, o fine-tuning total foi o melhor resultado. Treinar apenas a camada fc não foi suficiente, porque ficou bem abaixo das demais estratégias, enquanto o fine-tuning parcial trouxe ganhos claros em relação a isso. O fine-tuning total melhorou o desempenho e ficou estável no geral, sem sinais fortes de instabilidade.

A escolha dos pesos pré-treinados da ImageNet foi boa como experimento de transferência, mas não foi a mais ideal para obter o melhor resultado final, porque os datasets são bem diferentes em conteúdo e distribuição. Os resultados mostram justamente isso, já que quando há maior similaridade entre os domínios, a transferência tende a funcionar melhor, enquanto uma diferença grande entre os dados limita o ganho do pré-treinamento e aumenta a necessidade de adaptação das camadas do modelo.

Comparação entre as estratégias de treinamento:

Na estratégia do zero, a ResNet18 apresentou o melhor desempenho geral, com 99,24% de acurácia e F1-score no teste, além de curvas de perda caindo rapidamente e acurácia subindo até perto de 99%, o que indica bom aprendizado. Já em apenas a camada fc, o desempenho caiu bastante, para 76,42% de acurácia e 76,236% de F1-score, mostrando que treinar só a última camada foi insuficiente para adaptar bem o modelo. Nas curvas, a acurácia ficou bem mais baixa e o ganho ao

longo das épocas foi limitado.

No fine-tuning parcial, houve grande melhora, chegando a 97,1% de acurácia e 97,096% de F1-score, com curvas mais estáveis e boa aproximação entre treino e validação. No fine-tuning total, o desempenho ficou ainda mais próximo do treinamento do zero, com 98,98% de acurácia e F1-score, sugerindo melhor adaptação do modelo aos dados.

Em resumo, quanto mais camadas foram ajustadas, melhor foi o resultado, com o fine-tuning total quase alcançando o treino do zero.

Tabela com os resultados de acurácia e F1-score:

Estratégia	Camadas Treinadas	Acurácia no teste	F1-score no teste	Observações
ResNet18 do Zero	Todas	99.24	99.24	Melhor dentre os modelos (como esperado)
Apenas fc	fc	76.42	76.236	Pior dentre os modelos (como esperado)
Fine-tuning parcial	layer4 + fc	97.1	97.096	Curvas de custo e acurácia mais retas na validação
Fine-tuning total	layer1-layer4 + fc	98.98	98.98	Bem similar a treinar do zero

Gráficos de perda e acurácia:

Gráfico do modelo original:

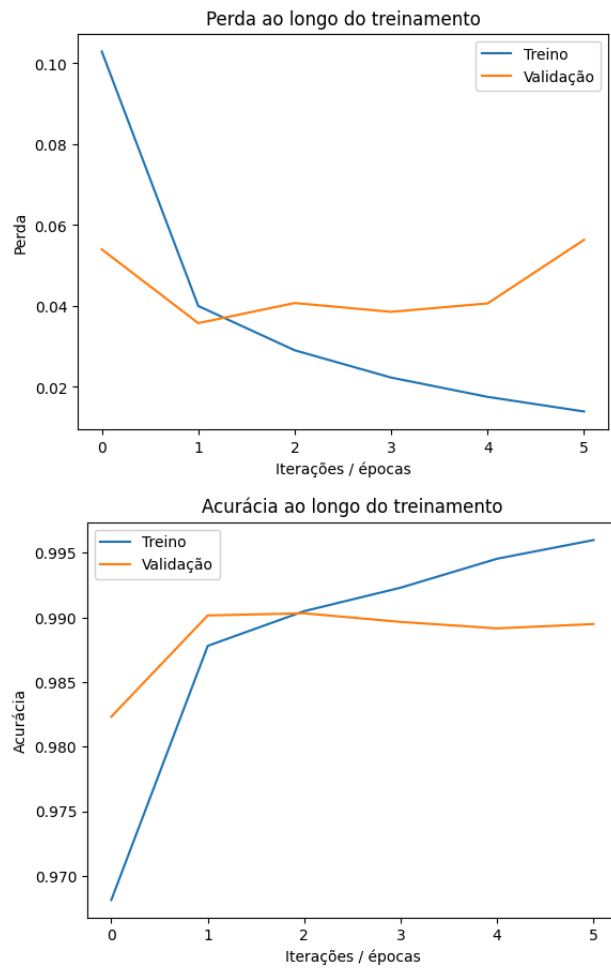


Gráfico do resnet18 sem pesos pré-treinados (exercício 1):

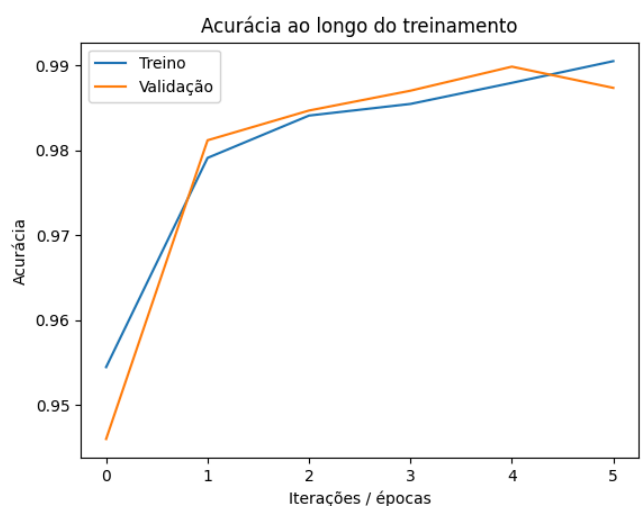
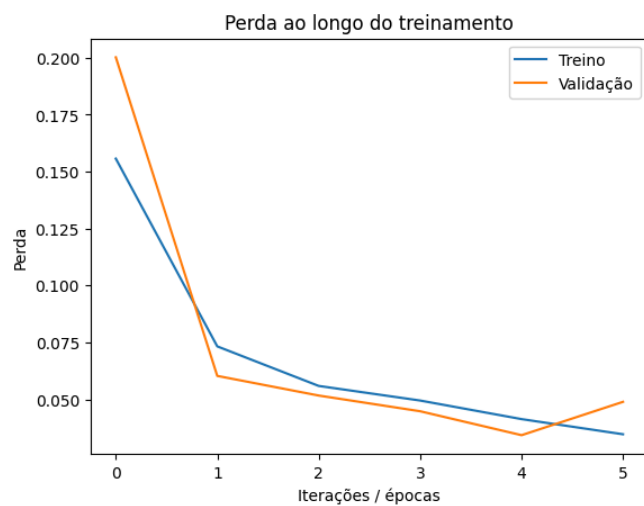


Gráfico do resnet18 com pesos pré-treinados, treinando apenas a última camada:

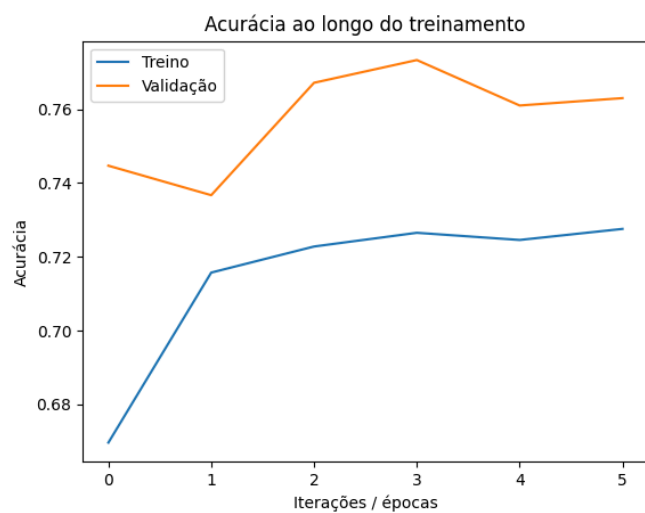
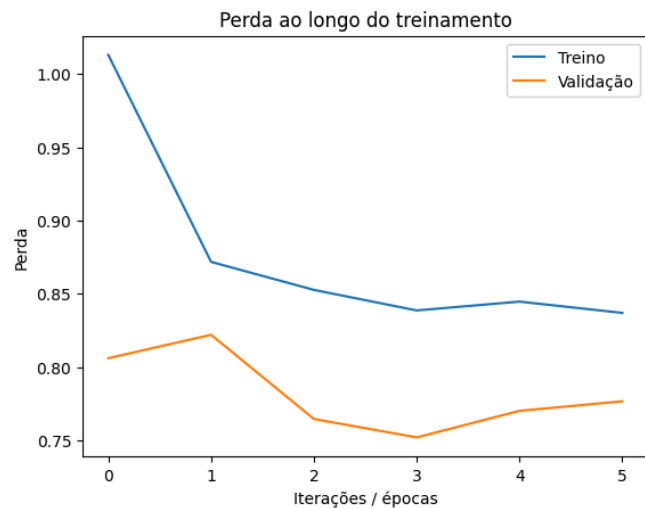


Gráfico do resnet18 com pesos pré-treinados com fine-tuning parcial:

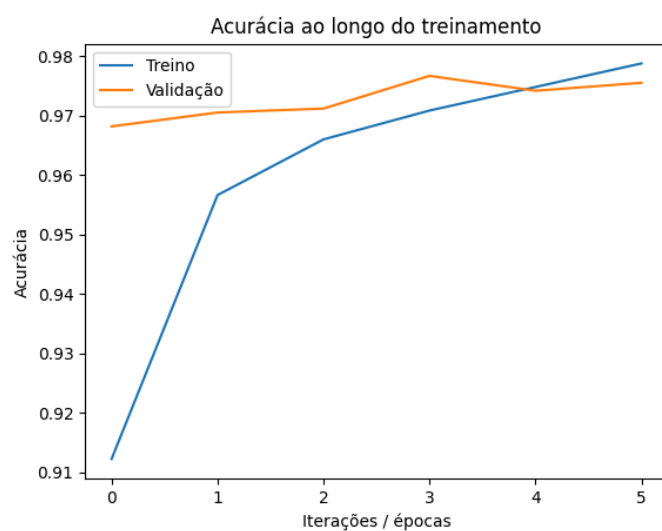
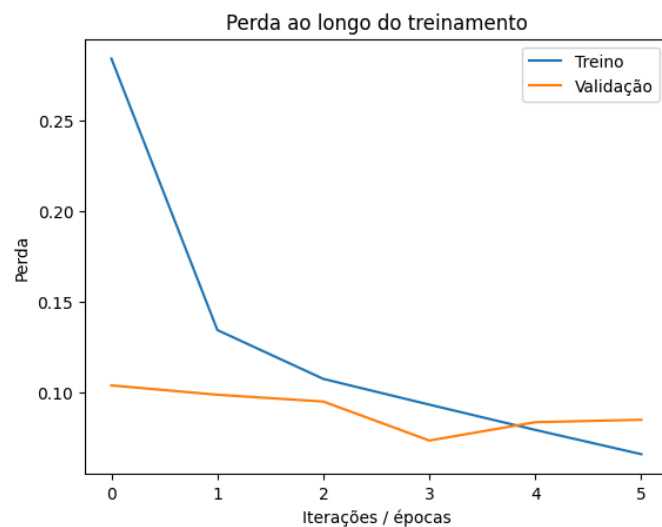
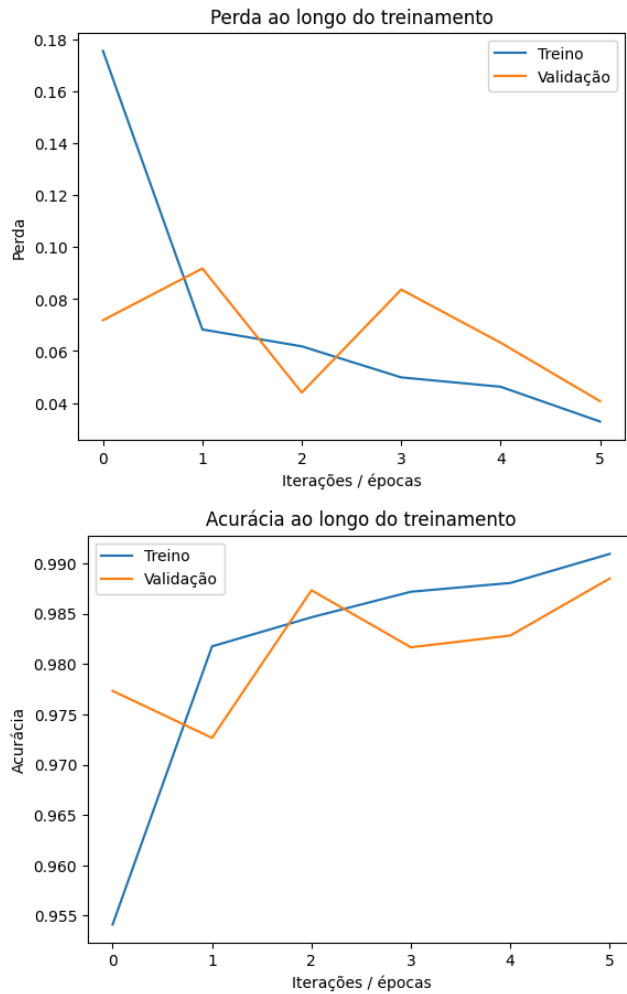


Gráfico do resnet18 com pesos pré-treinados com fine-tuning total:



Código:

[illegible]


```

scale=(0.9, 1.1),translate=(0.1, 0.1)),
                                transforms.RandomPerspective(),

transforms.Normalize((0.1307,), (0.3081,))]]

trans_basico = transforms.Compose([ transforms.ToTensor(),
                                transforms.RandomAffine(15,
translate=(0.1, 0.1)),
                                transforms.Normalize((0.1307,),
(0.3081,))])

trans_avancado = transforms.Compose([ transforms.TrivialAugmentWide(),
                                transforms.ToTensor(),
                                transforms.Normalize((0.1307,),
(0.3081,)),
                                transforms.RandomErasing()])

# Train/Test Datasets
train_dataset = torchvision.datasets.MNIST(root=root_path, train=True,
transform=trans, download=True)
test_dataset = torchvision.datasets.MNIST(root=root_path, train=False,
transform=trans)

# train_dataset = torchvision.datasets.MNIST(root=root_path,
train=True, transform=trans_aumentado, download=True)
# train_dataset = torchvision.datasets.MNIST(root=root_path,
train=True, transform=trans_basico, download=True)
# train_dataset = torchvision.datasets.MNIST(root=root_path,
train=True, transform=trans_avancado, download=True)
test_dataset = torchvision.datasets.MNIST(root=root_path, train=False,
transform=trans)

from torch.utils.data import DataLoader
from torch.utils.data import random_split
from torchvision.transforms import v2
from torch import Generator

batch_size=32
NUM_CLASSES = 100
SEED = 42

train_size = int(0.9 * len(train_dataset))
val_size = len(train_dataset) - train_size

```

```

train_subset, val_subset = random_split(train_dataset, [train_size,
val_size],

                                         generator =
Generator().manual_seed(SEED))
train_loader = DataLoader(dataset=train_subset, batch_size=batch_size,
shuffle=True)
val_loader = DataLoader(dataset=val_subset, batch_size=batch_size,
shuffle=False)
test_loader = DataLoader(dataset=test_dataset, batch_size=batch_size,
shuffle=False)

cutmix = v2.CutMix(num_classes=NUM_CLASSES)
mixup = v2.MixUp(num_classes=NUM_CLASSES)
cutmix_or_mixup = v2.RandomChoice([cutmix, mixup])

for images, labels in train_loader:
    images, labels = cutmix_or_mixup(images, labels)

import torch.nn as nn

class ConvNet(nn.Module):
    def __init__(self):
        super(ConvNet, self).__init__()
        self.layer1 = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=5, stride=1, padding=2),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2))
        self.layer2 = nn.Sequential(
            nn.Conv2d(32, 64, kernel_size=5, stride=1, padding=2),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2))
        self.fc1 = nn.Sequential(
            nn.Linear(7 * 7 * 64, 1000),
            nn.ReLU())
        self.fc2 = nn.Linear(1000, 10)

    def forward(self, x):
        out = self.layer1(x)
        out = self.layer2(out)
        out = out.reshape(out.size(0), -1)
        out = self.fc1(out)
        out = self.fc2(out)

```

```

        return out

# Instancia o Model()
model = ConvNet()

print(model)

# Modelo do ResNet18 do zero:
from torchvision.models import resnet18
from torchvision.models import ResNet18_Weights

# model = resnet18 # exercício 1
# model = resnet18()
model = resnet18(weights=ResNet18_Weights.IMAGENET1K_V1)

# Mudar número de classes na camada final, acomodando-o ao MNIST
model.fc = nn.Linear(model.fc.in_features, 10)

# Congelamento de todas as camadas:
for param in model.parameters():
    param.requires_grad = False

# Descongelamento das que forem ser treinadas:
for param in model.fc.parameters():
    param.requires_grad = True

for param in model.layer4.parameters():
    param.requires_grad = True

for param in model.parameters():
    param.requires_grad = True

print(model)

from torch.optim import Adam

# Hiperparâmetros
loss_fn = nn.CrossEntropyLoss()
learning_rate = 0.001
epochs = 6

# Instanciar o Otimizador Adam

```

```
optimizer = Adam(filter(lambda p: p.requires_grad, model.parameters()),
lr=learning_rate)

import torch

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

# Treinar o Modelo
total_step = len(train_loader) # quantos batches eu tenho

# Listas vazias
train_loss_list = []
train_acc_list = []
val_loss_list = []
val_acc_list = []

for epoch in range(epochs):
    model.train()
    train_loss = 0
    train_correct = 0
    train_total = 0
    for i, (images, labels) in enumerate(train_loader):
        # Gera a propagação (feed forward)
        images = images.to(device)
        labels = labels.to(device)
        outputs = model(images)

        # Calcula a função-custo
        loss = loss_fn(outputs, labels)
        train_loss += loss.item()

        # Retro-propagação (Backprop) e a otimização com Adam
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        # Acurácia
        total = labels.size(0)
        train_total += total
        _, predicted = torch.max(outputs.data, 1)
        correct = (predicted == labels).sum().item()
        train_correct += correct
```

```

    if (i + 1) % 100 == 0:
        print(f"Época [{epoch+1}/{epochs}], Step [{i+1}/{total_step}],
Custo: {round(loss.item(), 3)}, Acurácia: {round((correct / total) *
100, 3)}")

    avg_train_loss = train_loss / len(train_loader)
    train_acc = train_correct / train_total
    train_loss_list.append(avg_train_loss)
    train_acc_list.append(train_acc)
    model.eval()
    val_loss = 0
    val_correct = 0
    val_total = 0
    with torch.no_grad():
        for images, labels in val_loader:
            images = images.to(device)
            labels = labels.to(device)
            outputs = model(images)
            loss = loss_fn(outputs, labels)
            val_loss += loss.item()
            _, predicted = torch.max(outputs.data, 1)
            val_total += labels.size(0)
            val_correct += (predicted == labels).sum().item()
    avg_val_loss = val_loss / len(val_loader)
    val_acc = val_correct / val_total
    val_loss_list.append(avg_val_loss)
    val_acc_list.append(val_acc)
    print(f"Validação - Perda: {round(avg_val_loss, 3)}, Acurácia:
{round(val_acc * 100, 3)}")

import matplotlib.pyplot as plt

plt.figure()
plt.plot(train_loss_list, label="Treino")
plt.plot(val_loss_list, label="Validação")
plt.xlabel("Iterações / épocas")
plt.ylabel("Perda")
plt.title("Perda ao longo do treinamento")
plt.legend()
plt.show()

plt.figure()
plt.plot(train_acc_list, label="Treino")

```

```

plt.plot(val_acc_list, label="Validação")
plt.xlabel("Iterações / épocas")
plt.ylabel("Acurácia")
plt.title("Acurácia ao longo do treinamento")
plt.legend()
plt.show()

from sklearn.metrics import f1_score

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

model.eval() # coloca o modelo em modo de avaliação (sem calcular
              # gradientes)

with torch.no_grad():
    correct = 0
    total = 0
    toda_predictions = []
    toda_labels = []

    for images, labels in test_loader:
        # Feed-forward com as imagens de teste
        images = images.to(device)
        labels = labels.to(device)
        outputs = model(images)

        # gera previsões usando a função max()
        _, predicted = torch.max(outputs.data, 1)

        # Acumula total e corretas
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

        # Guarda para calcular o F1-score
        toda_predictions.extend(predicted.cpu().numpy())
        toda_labels.extend(labels.cpu().numpy())

    accuracy = (correct / total) * 100
    f1 = f1_score(toda_labels, toda_predictions, average="weighted") *

```

```
    print(f"Acurácia do Modelo em 10k imagens de teste:  
{round(accuracy, 3)}")  
    print(f"F1-score do Modelo em 10k imagens de teste: {round(f1,  
3)}")
```